



Probeklausur - Informatik (INF) 1100 - Fortgeschrittene Programmierkonzepte(FPK)

Datum: 19.12.2024	Dauer: 90 Minuten	Material: Ein Buch mit ISBN-Nr
-------------------	-------------------	--------------------------------

Name:

Matr.-Nr.:

Viel Erfolg!

Hinweise:

1. Die Heftklammern dürfen nicht gelöst werden. Bitte überprüfen Sie: Die Klausur umfasst **14 Seiten incl. Deckblatt und Arbeitsblätter**. Die Seiten sind beidseitig bedruckt.
2. Bearbeiten Sie die Fragen direkt in der Angabe. Nutzen Sie ggfs. die Arbeitsblätter und Rückseiten.
3. Sollten Ihrer Meinung nach Widersprüche in den Aufgaben existieren bzw. Angaben fehlen, so machen Sie **sinnvolle Annahmen und dokumentieren Sie diese**.
4. Die Punkteverteilung dient zur Orientierung, sie ist jedoch unverbindlich.
5. Alle Fragen beziehen sich auf die Programmiersprache Java; Ausnahmen sind gekennzeichnet.
6. Bitte schreiben Sie nicht mit Bleistift, roten oder grünen Stiften und wenn möglich **leserlich**.

MUSTERLÖSUNG gibt es im neuen Jahr!

1. Aufgabe - Allgemeines**8+6 Punkte****a)**

Markieren Sie die richtige Antwort bzw. Aussage; pro Frage ist genau eine Antwort zu markieren.

1. Interfaces und abstrakte Klassen in Java 9 und neuer.
 - ☐ Eine abstrakte Klasse **muss mindestens eine** abstrakte Methode haben.
 - ☐ Ererbte abstrakte Methoden müssen **immer** implementiert werden.
 - ☒ Methoden in Interfaces können **private** sein.
2. Bezüglich innerer Klassen gilt:
 - ☐ Innere Klassen können keine Schnittstellen implementieren.
 - ☐ Innere Klassen müssen immer als **static** deklariert sein.
 - ☒ Es gibt sowohl innere Klassen als auch innere Interfaces.
3. Welche der folgenden Signaturen ist korrekt und generisch?
 - ☒ `abstract <T> void a(T t);`
 - ☐ `abstract void a(T t);`
 - ☐ `<T> abstract void a(T t);`
4. Bezüglich Ausnahmen (Exception) gilt:
 - ☐ Ungeprüfte Ausnahmen müssen in der Methode behandelt werden, in der sie auftreten.
 - ☒ Geprüfte Ausnahmen müssen lokal behandelt oder mit **throws** ausgewiesen werden.
 - ☐ Eine Ausnahme muss immer mit `try..catch` behandelt werden.
5. Bezüglich Sichtbarkeiten gilt:
 - ☐ Interfaces können **protected** Methoden enthalten.
 - ☐ Innere Klassen ohne Sichtbarkeitsangabe sind öffentlich sichtbar.
 - ☒ Ist eine innere Klasse **private**, so kann sie in abgeleiteten Klassen nicht instanziiert werden.

6. Bezüglich funktionaler Programmierung gilt:
- ✓ *Endrekursiv* bedeutet dass der Rekursionsschritt die letzte Anweisung ist.
 - ☐ Rufen zwei Methoden **f** und **g** sich wechselseitig gegenseitig auf, so spricht man von *kaskadierter* Rekursion.
 - ☐ Eine Rekursion kann auch ohne Terminalfall regulär berechnet werden.
7. Bezüglich paralleler (nebenläufiger) Ausführung gilt:
- ☐ Ein *kritischer Abschnitt* ist ein Teil einer Methode, welcher besonders kompliziert ist.
 - ✓ Das Java Interface **Future** wird für asynchrone Programmierung verwendet.
 - ☐ Methoden welche nur von genau einem Thread gleichzeitig ausgeführt werden dürfen, müssen mit **@Synchronized** annotiert werden.
8. Bezüglich paralleler Verarbeitung gilt:
- ☐ Java regelt konkurrierenden Zugriff automatisch, wodurch Deadlocks vermieden werden.
 - ☐ Das Gegenstück zu **wait()** ist **signal()**.
 - ✓ Die Methode **notify()** kann nur in kritischen Abschnitten und auf dem Lock-objekt verwendet werden.

b)

Beantworten Sie folgende Fragen kurz und knapp (je 2 Punkte):

1. Nennen Sie zwei syntaktische Alternativen zu einer anonymen inneren Klasse, die nur eine Methode hat (@FunctionalInterface).

Lösung:

Lambdalausdruck, Methodenreferenz

2. Wozu dient die Annotation @Deprecated?

Lösung:

Markiert eine Methode oder Klasse, dass diese in zukünftigen Versionen ersetzt wird oder ganz herausfällt.

3. Ordnen Sie die Designpatterns ihrer Kurzbeschreibung zu?

Name:

Matrikelnr.:

Lösung:

Factory ----- Erzeugung von Objekten

Singleton ----- Eine globale Instanz

Fliegengewicht ---- Reduktion des Speicherbedarfs

Visitor ----- Traversieren von Datenstrukturen

2. Aufgabe - Generics**5+3+1+4 Punkte****a)**

Schreiben Sie eine generische Klasse **Container**, welche Objekte eines beliebigen (aber festen) Typs speichert. Die Klasse soll weiterhin eine öffentliche Methode besitzen, welche den Laufzeittyp des gespeicherten Elements zurückgibt oder **null** wenn das Element **null** ist.

Lösung:

```
// Klasse Container
public class Container<T> {

    // Attribute
    private T obj;

    // Konstruktor
    public Container(T obj) {
        this.obj = obj;
    }

    // Methode getContainedClass
    public Class getContainedClass() {
        if (obj == null)
            return null;
        return obj.getClass();
    }

    public static void main(String[] args) {
        Container<Integer> c = new Container<>();
        c.obj = 2;
        System.out.println(c.getContainedClass());
    }
}
```

b)

Gegeben sei die folgende (nicht-generische) Methodensignatur:

`Comparable minimum(Comparable[] feld)`

Schreiben sie eine generische Variante dieser Signatur, welche es erlaubt ein Array eines festen Typs unter Verwendung der Methode `Comparable.compareTo` zu sortieren.

Lösung:

```
static <T extends Comparable<? super T>> T minimum(T[] feld)
```

Name:

Matrikelnr.:

c)

Wie heisst der Mechanismus in Java um den Objekttyp zur Laufzeit zu bestimmen?

Lösung:

Reflection.

d)

Kurz und knapp: Was bedeuten die Zeichen ? und & in Zusammenhang mit Generics?

Lösung:

? ist wildcard, beliebige aber feste Klasse (2P)

& Auflistungsoperator für komplexes Bound (2P)

3. Aufgabe - Generics**5+5 Punkte****a)**

Schreiben Sie eine statische, generische Methode `maximum`. Diese bekommt bekommt n Parameter vom Typ `T` übergeben und soll darin das Maximum finden. Die Methode hat folgende 2 Einschränkungen:

- Der Vergleich soll über `Comparable` erfolgen
- Die Parameter sollen nur numerisch sein, sprich sie sollen vom Typ `Number` sein.

Schränken Sie also den Typ `T` mit entsprechenden Bounds ein!

Zur Hilfe dient folgendes Programm, dass lauffähig sein sollte und entsprechende Resultate produziert:

Lösung:

```
public class Generics1 {

    public static void main(String[] args) {
        // Integer
        System.out.println("Max is: " + maximum(3, 4, 5));
        // Max is : 5

        // Double
        System.out.println("Max is: " + maximum(6.6, 8.8, 7.7));
        // Max is : 8.8
    }

    public static <T extends Number & Comparable<T>> T maximum(T ... xs)
    {
        assert(xs.length > 0);
        T max = xs[0];
        for (int i = 0; i < xs.length; i++) {
            if (xs[i].compareTo(max) > 0)
                max = xs[i];
        }
        return max;
    }
}
```

Name:

Matrikelnr.:

b)

Schreiben Sie eine generische Klasse `Pair`. Diese hat genau 2 Attribute `t` und `s`, wobei `t` vom Typ `T` sein soll und `s` eine iterierbare Liste vom `S` sein soll. Die Attribute haben die folgenden 2 Einschränkungen:

- Für `t` gilt, dass der Typ `Comparable` sein muss
- Für `s` gilt, dass der Typ `Iterable` sein muss

Schränken Sie also die Klasse mit entsprechenden Bounds ein!

Vervollständigen Sie die Implementierung der Klasse `Pair`.

Lösung:

```
public class Generics2 {  
    static class Pair<T extends Comparable<T>, S extends Iterable<?>> {  
        private T t;  
        private S s;  
  
        public Pair(T t, S s) {  
            this.t = t;  
            this.s = s;  
        }  
  
        @Override  
        public String toString() {  
            return "Pair{" + t + ":\u0020" + s + "}";  
        }  
    }  
  
    public static void main(String[] args) {  
        System.out.println(new Pair<Integer, List<String>>>(1, Arrays.asList("A", "B", "C")));  
  
        // Pair{1: [A, B, C]}  
        System.out.println(new Pair<String, List<Integer>>>("A", Arrays.asList(1, 2, 3, 4)));  
        // Pair{A: [1, 2, 3, 4]}  
    }  
}
```


4. Aufgabe - Design Pattern**4+4+4 Punkte****a)**

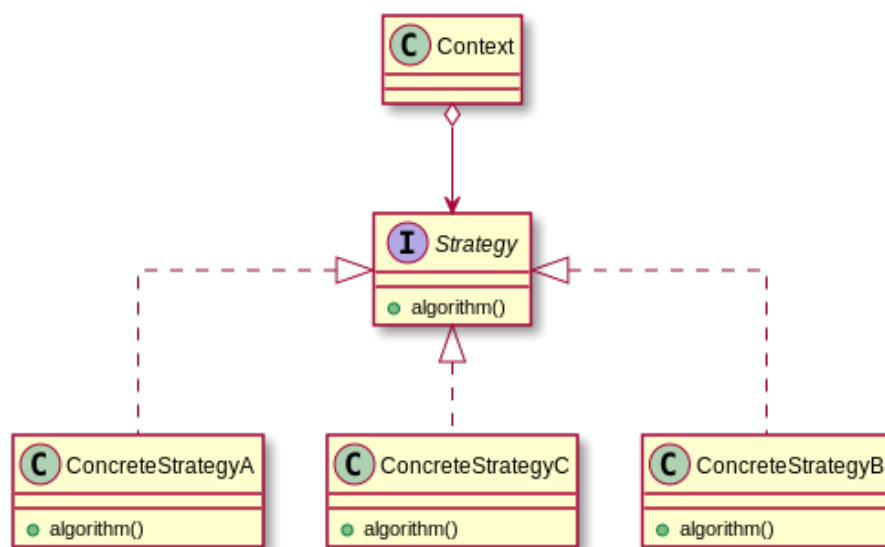
Kurz und knapp: Was ist der Sinn des Strategiemusters (strategy pattern).

Lösung:

Definiert eine Familie von Algorithmen und kapselt diese. Dadurch sind die einzelnen Strategien austauschbar. Das Strategiemuster ermöglicht es, den Algorithmus unabhängig von nutzenden Klienten zu variieren.

b)

Zeichnen Sie das Klassendiagramm des Strategiemusters.

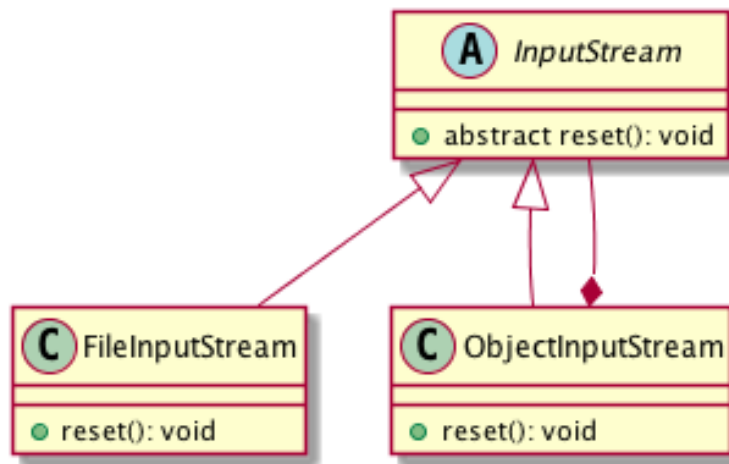
Lösung:

Name:

Matrikelnr.:

c)

Benennen Sie das folgende Pattern und erläutern Sie kurz einen Anwendungsfall.



Lösung:

Decorator:

Das Decorator Design Pattern ermöglicht das dynamische Hinzufügen von Fähigkeiten zu einer Klasse. Dazu wird die Klasse, dessen Verhalten wir erweitern möchten, mit anderen Klassen (Decorator, Dekorierer) dekoriert. Das heißt der Decorator umschließt (enthält) die Component. Der Decorator ist vom selben Typ wie das zudekorierende Objekt, hat somit die gleiche Schnittstelle und kann an der selben Stelle wie die Component benutzt werden. Er delegiert Methodenaufrufe an seine Component weiter und führt sein eigenes Verhalten davor oder danach aus.

Anwendungsfälle: Bei der Speicherung von Textdaten können zusätzliche Features, wie das Maskieren von Umlauten oder ein Komprimierungsalgorithmus hinzugeschaltet werden.

Wenn Funktionalitätserweiterung mittels Vererbung impraktikabel ist: Dies ist zum einen der Fall bei voneinander unabhängigen Erweiterungen, wenn unter Beachtung jeder möglichen Erweiterungskombination eine schier unüberschaubare Anzahl von Klassen entstehen würde.

Das klassische Kaffeebeispiel: Es sind 3 Kaffeegrundsorten (Espresso, Edelfrucht, Cappuccino, Latte Macchiato) und 4 optionale Zusätze (Milch, Schoko, Zucker, Sahne) gegeben. Das ergibt 16 Subklassen! Unüberschaubar, unwartbar und unflexibel.

Optional ginge evtl. auch **Composite Pattern**!

5. Aufgabe - Threads**10 Punkte**

Der folgende Codeausschnitt soll einen threadsicheren Buffer für ein Consumer-Producer-Problem implementieren. Ergänzen Sie den Quelltext an den mit Platzhaltern (____) markierten Stellen (nicht alle Leerstellen müssen befüllt werden), so dass der Buffer sich wie erwartet verhält, und zwar:

- in `get()` wartet, bis mindestens 1 Element im Buffer verfügbar ist.
- in `put()` wartet, bis mindestens 1 Element im Buffer frei ist
- eine Verklemmung (deadlock) vermeidet.

Lösung:

```
public class Buffer<T> {  
  
    private Queue<T> queue = new LinkedList<>();  
    private final int maxSize = 10;  
  
    public T get() throws Exception{  
        synchronized (this) {  
            while (queue.size() == 0)  
                wait();  
  
            T obj = queue.remove();  
            notify();  
            return obj;  
        }  
    }  
  
    public void put(T obj) throws Exception {  
        synchronized (this) {  
            while (queue.size() >= maxSize)  
                wait();  
            queue.add(obj);  
            notify();  
        }  
    }  
}
```

6. Aufgabe - Functional Interfaces**5+5+3 Punkte**

Gegeben ist das Interface `BinaryOperator<T extends Comparable>` mit der `apply`-Methode:

```
@FunctionalInterface
interface BinaryOperator<T extends Comparable> {
    T apply(T a, T b);
}
```

Die `apply`-Methode bekommt 2 Parameter vom Typ `T` und gibt einen Wert vom Typ `T` zurück.

a)

Schreiben Sie eine Methode `reduce`, die die Methode `apply` auf jedes Element einer übergebenen Liste vom Typ `T` anwendet. Stellen Sie sich vor, dass sie eine Liste von Zahlen das Maximum ermitteln wollen. Der Code dazu könnte wie folgt aussehen:

```
interface BinaryOperator<T extends Comparable> {
    T apply(T a, T b);
}

static <T extends Comparable> T reduce(List<T> list, BinaryOperator<T> func
) {}

public static void main(String[] args) {

    java.util.List<Integer> l1 = Arrays.asList(29, 19, 20, 21, 25, 22, 23);

    Integer max = reduce( l1, new BinaryOperator<Integer>() {
        @Override
        public Integer apply(Integer a, Integer b) {
            return Integer.max(a,b);
        }
    });
    System.out.println(max);
}
```

In dem Code-Beispiel würde die `reduce`-Methode nun 29 zurückgeben.

Implementieren Sie die vorgegebene `reduce`-Methode nun so, dass das möglich ist unter Verwendung des `BinaryOperators`. Sie können davon ausgehen, dass die übergebene Liste mindestens 1 Element enthält!

Die Signatur der Methode ist:

```
static <T extends Comparable> T reduce(List<T> l, BinaryOperator<T> f)
```

Lösung:

```

iterativ:
static <T extends Comparable> T reduce(List<T> l, BinaryOperator<T> f) {
    T result = l.get(0);
    for (T elem: l) {
        result = f.apply(elem, result);
    }
    return result;
}

rekursiv:
static <T extends Comparable> T reduceR(List<T> l, BinaryOperator<T> f)
{
    if (l.size() == 1) return l.get(0);
    return f.apply(l.get(0), reduceR(l.subList(1, l.size()),f));
}

```

b)

Gehen Sie davon aus, dass die `reduce`-Methode existiert. Nun sollen Sie das Minimum in einer Liste von Strings bestimmen. Hierzu bietet es sich nun an, die `reduce`-Methode zu verwenden.

Schreiben sie also eine `BinaryOperator`-Implementierung (ähnlich der Implementierung und a) unter Verwendung der Boundry `T extends Comparable` als anonyme innere Klasse.

Im Prinzip sollte folgendes Programm funktionieren:

Lösung:

```

public static void main(String[] args) {

    List<String> l2 = Arrays.asList("das", "ist", "ein", "test");

    String ms = reduce(l2, new BinaryOperator<String>() {
        @Override
        public String apply(String a, String b) {
            return (a.compareTo(b) < 0 ? a : b);
        }
    });
    System.out.println(ms);
}

```

c)

Schreiben Sie die anonymen inneren Klasse aus Teilaufgabe b als Lambda-Ausdruck:

Name:

Matrikelnr.:

Lösung:

```
public static void main(String[] args) {  
    java.util.List<Integer> l1 = Arrays.asList(29, 19, 20, 21, 25, 22,  
        23);  
  
    Integer max = reduce( l1, (a,b) -> Integer.max(a,b));  
    System.out.println(max);  
}
```